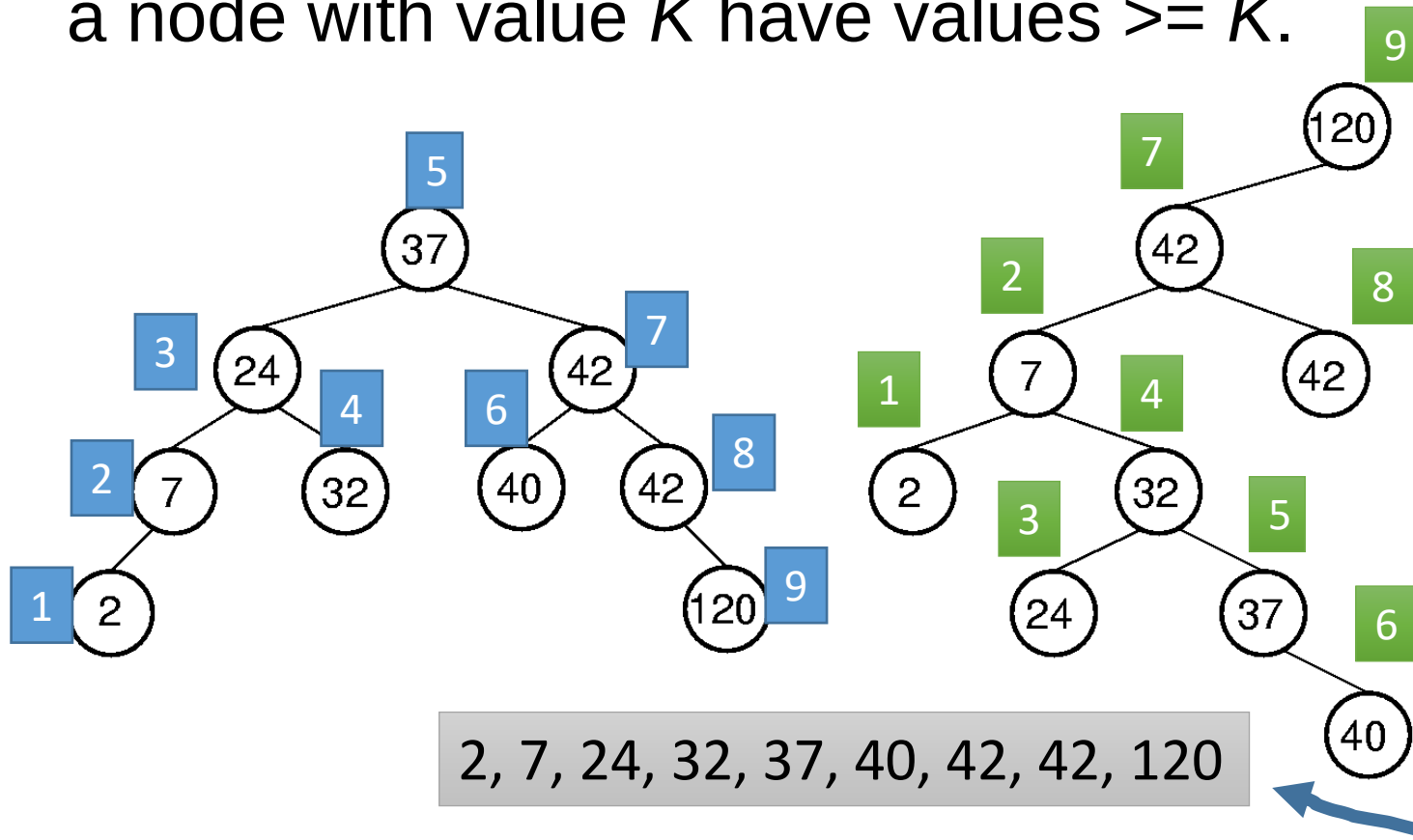


Binary Search Trees

Binary Search Trees

BST Property: All elements stored in the left subtree of a node with value K have values $< K$. All elements stored in the right subtree of a node with value K have values $\geq K$.



What happens on an inorder traversal on a BST?



The resulting enumeration will be in sorted order (low to high).

BST Implementation

- We first look at the Dictionary ADT which the BST implementation is based on.

136

Chap. 4 Lists, Stacks, and Queues

```
// The Dictionary abstract class.
template <typename Key, typename E>
class Dictionary {
private:
    void operator =(const Dictionary&) {}
    Dictionary(const Dictionary&) {}

public:
    Dictionary() {} // Default constructor
    virtual ~Dictionary() {} // Base destructor

    // Reinitialize dictionary
    virtual void clear() = 0;

    // Insert a record
    // k: The key for the record being inserted.
    // e: The record being inserted.
    virtual void insert(const Key& k, const E& e) = 0;

    // Remove and return a record.
    // k: The key of the record to be removed.
    // Return: A matching record. If multiple records match
    // "k", remove an arbitrary one. Return NULL if no record
    // with key "k" exists.
    virtual E remove(const Key& k) = 0;

    // Remove and return an arbitrary record from dictionary.
    // Return: The record removed, or NULL if none exists.
    virtual E removeAny() = 0;

    // Return: A record matching "k" (NULL if none exists).
    // If multiple records match, return an arbitrary one.
    // k: The key of the record to find
    virtual E find(const Key& k) const = 0;

    // Return the number of records in the dictionary.
    virtual int size() = 0;
};
```

Figure 4.28 The ADT for a simple dictionary.

Binary Tree Node Class

We studied It earlier...

156

Chap. 5 Binary Trees

```
// Binary tree node abstract class
template <typename E> class BinNode {
public:
    virtual ~BinNode() {} // Base destructor

    // Return the node's value
    virtual E& element() = 0;

    // Set the node's value
    virtual void setElement(const E&) = 0;

    // Return the node's left child
    virtual BinNode* left() const = 0;

    // Set the node's left child
    virtual void setLeft(BinNode*) = 0;

    // Return the node's right child
    virtual BinNode* right() const = 0;

    // Set the node's right child
    virtual void setRight(BinNode*) = 0;

    // Return true if the node is a leaf, false otherwise
    virtual bool isLeaf() = 0;
};
```

Figure 5.5 A binary tree node ADT.

Example 5.1 The preorder enumeration for the tree of Figure 5.1 is

ABDCEGFHL

The first node printed is the root. Then all nodes of the left subtree are printed (in preorder) before any node of the right subtree.

Alternatively, we might wish to visit each node *only after* we visit its children (and their subtrees). For example, this would be necessary if we wish to return all nodes in the tree to free store. We would like to delete the children of a node before deleting the node itself. But to do that requires that the children's children be deleted first, and so on. This is called a **postorder traversal**.

Example 5.2 The postorder enumeration for the tree of Figure 5.1 is

DBGEHIFCA

An **inorder traversal** first visits the left child (including its entire subtree), then visits the node, and finally visits the right child (including its entire subtree). The

BST Node: We studied it earlier...

Sec. 5.3 Binary Tree Node Implementations

161

```
// Simple binary tree node implementation
template <typename Key, typename E>
class BSTNode : public BinNode<E> {
private:
    Key k;           // The node's key
    E it;            // The node's value
    BSTNode* lc;      // Pointer to left child
    BSTNode* rc;      // Pointer to right child

public:
    // Two constructors -- with and without initial values
    BSTNode() { lc = rc = NULL; }
    BSTNode(Key K, E e, BSTNode* l = NULL, BSTNode* r = NULL)
    { k = K; it = e; lc = l; rc = r; }
    ~BSTNode() {} // Destructor

    // Functions to set and return the value and key
    E& element() { return it; }
    void setElement(const E& e) { it = e; }
    Key& key() { return k; }
    void setKey(const Key& K) { k = K; }

    // Functions to set and return the children
    inline BSTNode* left() const { return lc; }
    void setLeft(BinNode<E>* b) { lc = (BSTNode*)b; }
    inline BSTNode* right() const { return rc; }
    void setRight(BinNode<E>* b) { rc = (BSTNode*)b; }

    // Return true if it is a leaf, false otherwise
    bool isLeaf() { return (lc == NULL) && (rc == NULL); }
};
```

Figure 5.7 A binary tree node class implementation.

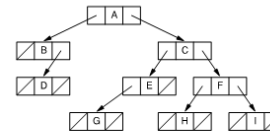


Figure 5.8 Illustration of a typical pointer-based binary tree implementation, where each node stores two child pointers and a value.

BST Implementation (on Dictionary ADT)

```
// Binary Search Tree implementation for the Dictionary ADT
template <typename Key, typename E>
class BST : public Dictionary<Key,E> {
private:
    BSTNode<Key,E>* root; // Root of the BST
    int nodecount; // Number of nodes in the BST

    // Private "helper" functions
    void clearhelp(BSTNode<Key, E>*);
    BSTNode<Key,E>* inserthelp(BSTNode<Key, E>*,
                               const Key&, const E&);
    BSTNode<Key,E>* deletemin(BSTNode<Key, E>*);
    BSTNode<Key,E>* getmin(BSTNode<Key, E>*);
    BSTNode<Key,E>* removehelp(BSTNode<Key, E>*, const Key&);
    E findhelp(BSTNode<Key, E>*, const Key&) const;
    void printhelp(BSTNode<Key, E>*, int) const;

public:
    BST() { root = NULL; nodecount = 0; } // Constructor
    ~BST() { clearhelp(root); } // Destructor

    void clear() // Reinitialize tree
    { clearhelp(root); root = NULL; nodecount = 0; }

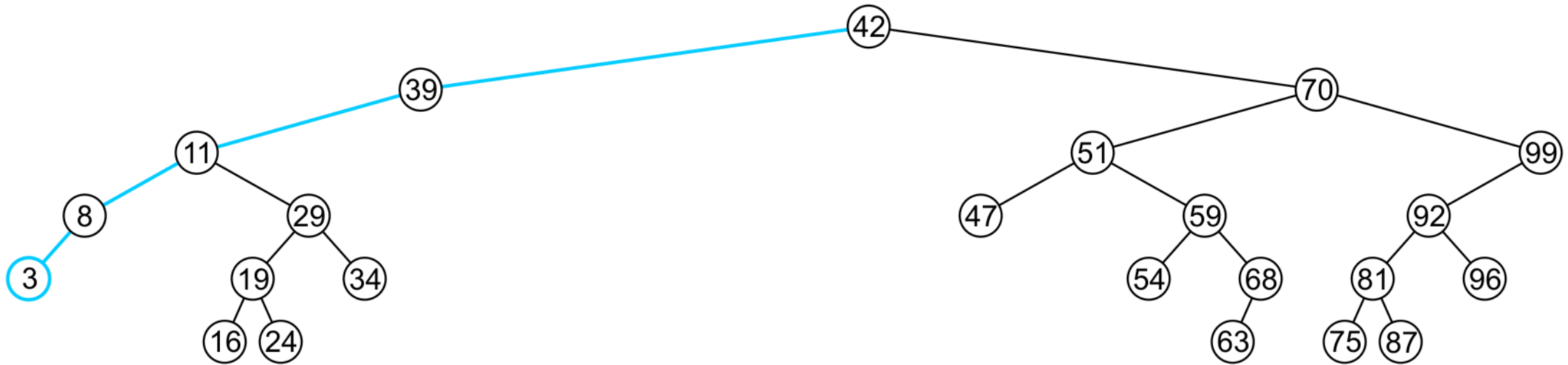
    // Insert a record into the tree.
    // k Key value of the record.
    // e The record to insert.
    void insert(const Key& k, const E& e) {
        root = inserthelp(root, k, e);
        nodecount++;
    }

    // Remove a record from the tree.
    // k Key value of record to remove.
    // Return: The record removed, or NULL if there is none.
    E remove(const Key& k) {
        E temp = findhelp(root, k); // First find it
        if (temp != NULL) {
            root = removehelp(root, k);
            nodecount--;
        }
        return temp;
    }
}
```

Figure 5.14 The binary search tree implementation.

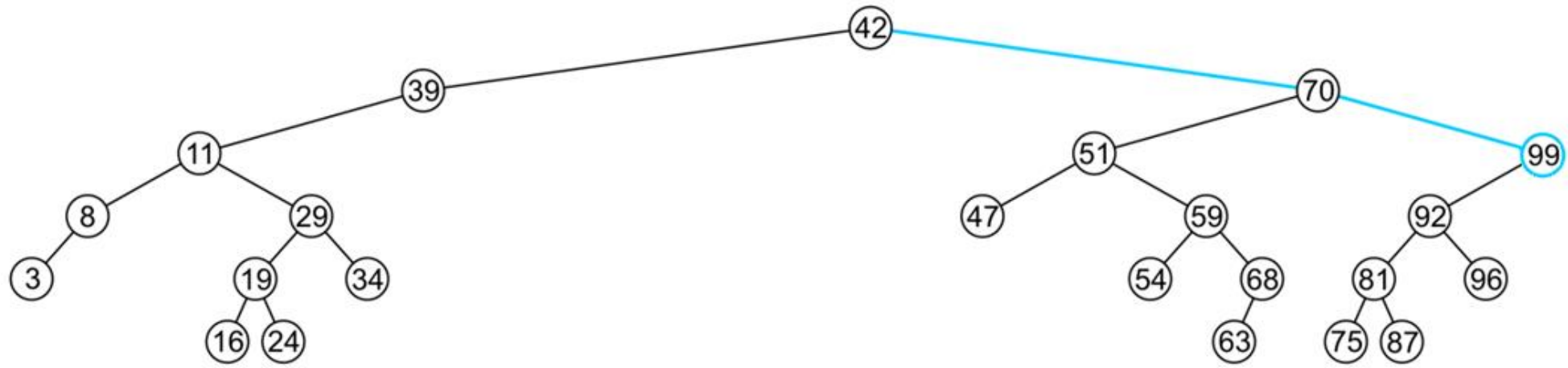
BST methods: Including a
conceptual view

Finding the Minimum Object



```
template <typename Key, typename E>
BSTNode<Key, E>* BST<Key, E>::
getmin(BSTNode<Key, E>* rt) {
    if (rt->left() == NULL)
        return rt;
    else return getmin(rt->left());
}
```

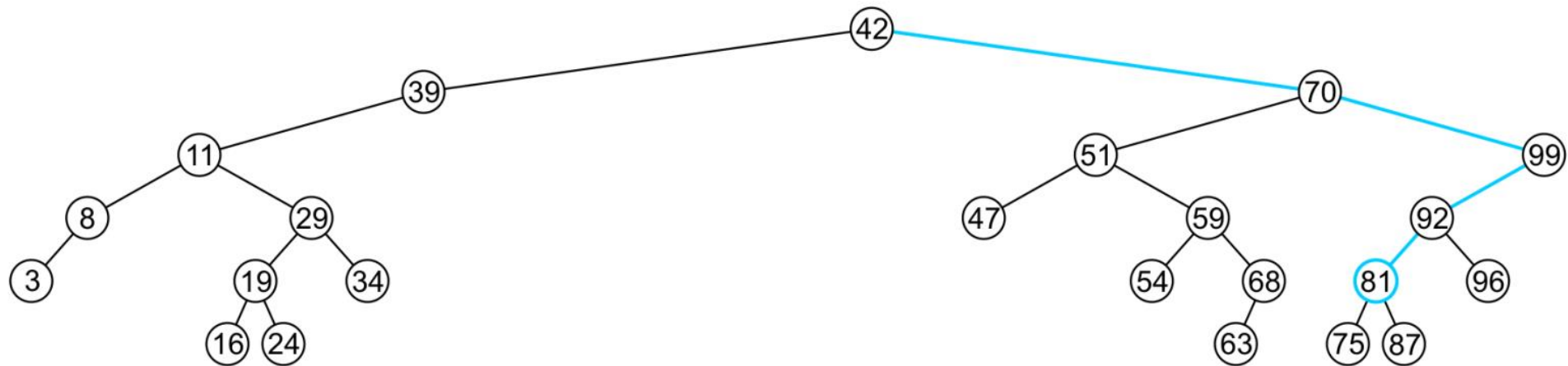

Finding the Maximum Object



The extreme values are not necessarily leaf nodes

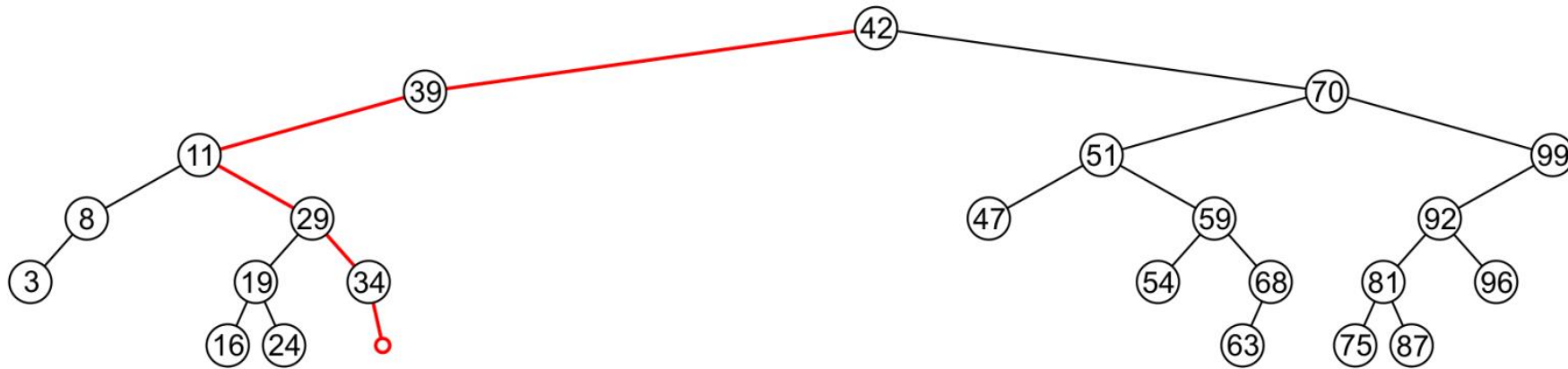
Search/Find

Finding the value 81



Search/Find

Finding a non-existent value: 36



BST Search

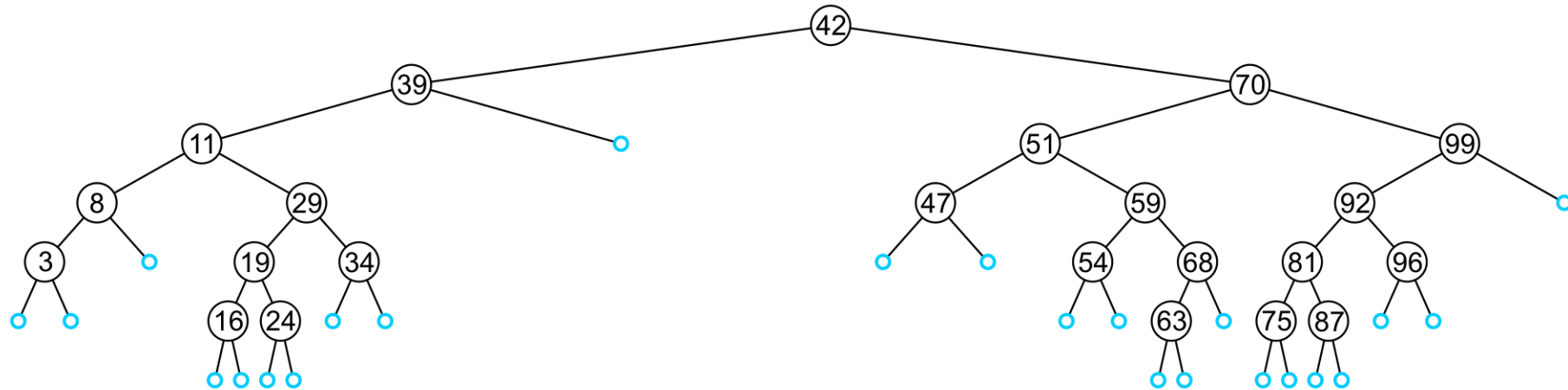
```
template <typename Key, typename E>
E BST<Key, E>::findhelp(BSTNode<Key, E>* root,
                        const Key& k) const {
    if (root == NULL) return NULL;           // Empty tree
    if (k < root->key())
        return findhelp(root->left(), k);   // Check left
    else if (k > root->key())
        return findhelp(root->right(), k);  // Check right
    else return root->element();             // Found it
}

// Return Record with key value k, NULL if none exist.
// k: The key value to find. */
// Return some record matching "k".
// Return true if such exists, false otherwise. If
// multiple records match "k", return an arbitrary one.
E find(const Key& k) const { return findhelp(root, k); }
```

Insert

An insertion will be performed at a leaf node:

- Any empty node is a possible location for an insertion

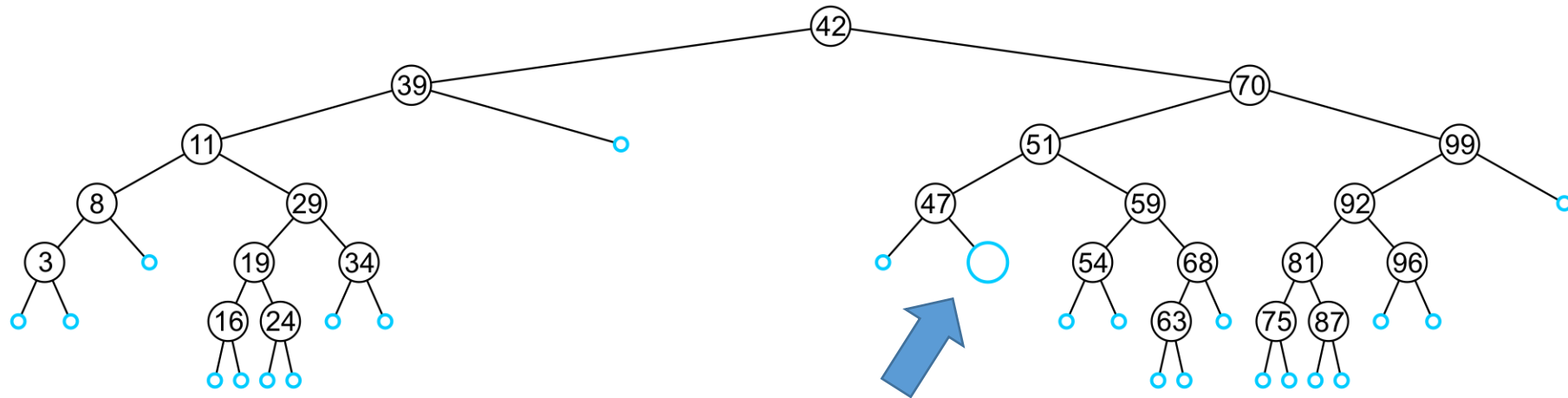


The values which may be inserted at any empty node depend on the surrounding nodes

Insert

For example, this node may hold ???

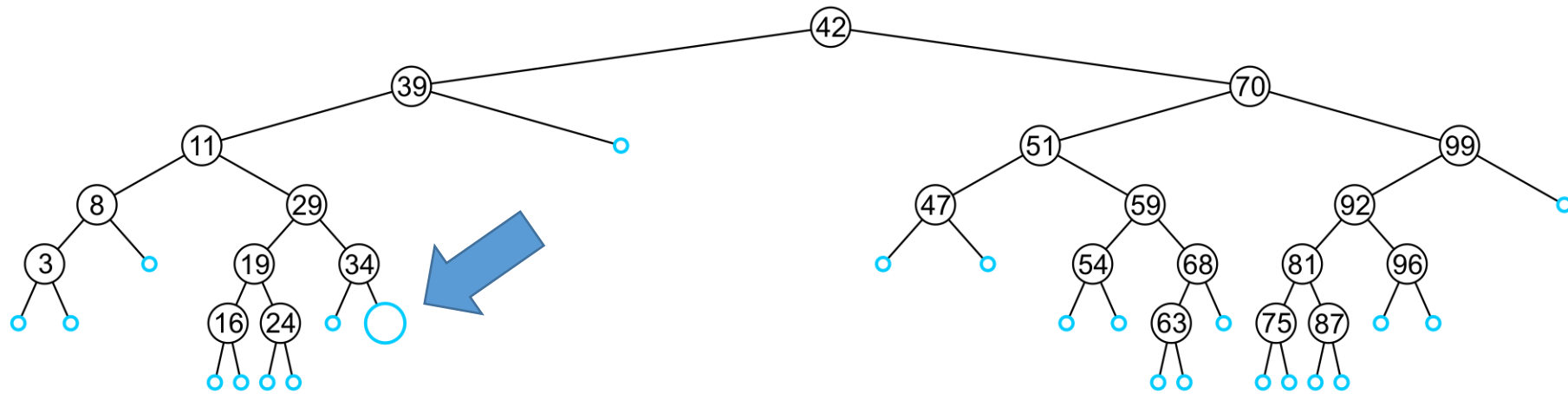
48, 49, or 50



Insert

An insertion at this location must be ???

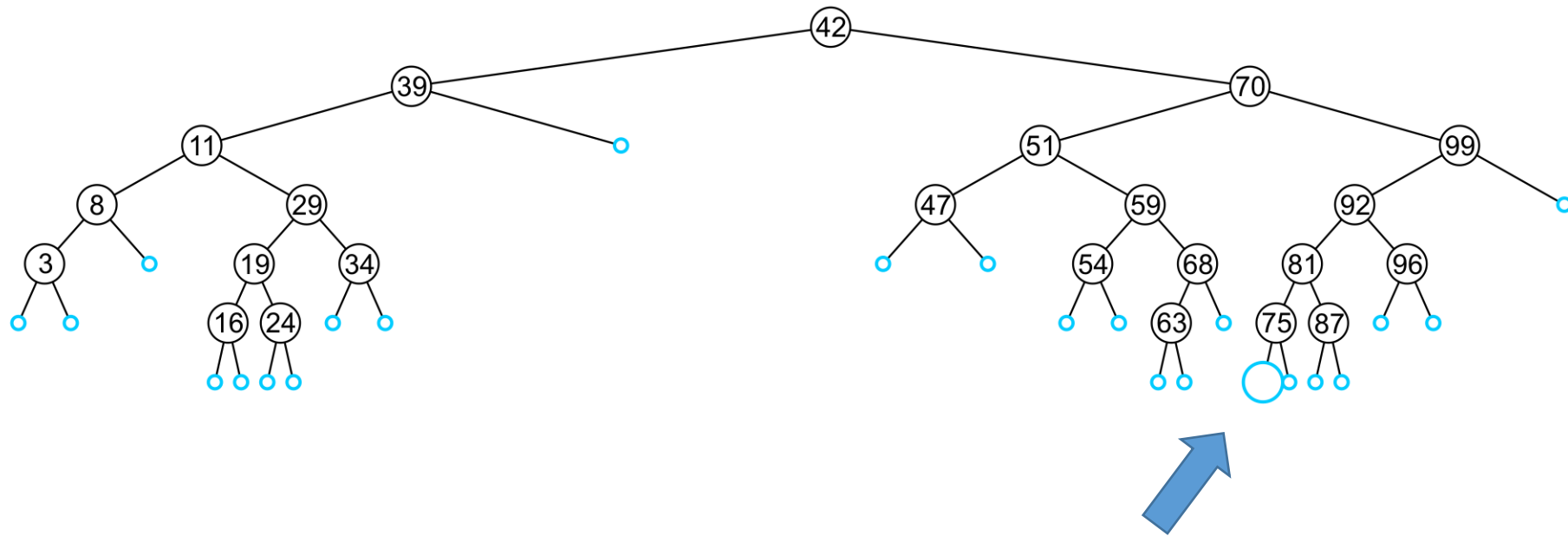
35, 36, 37, or 38



Insert

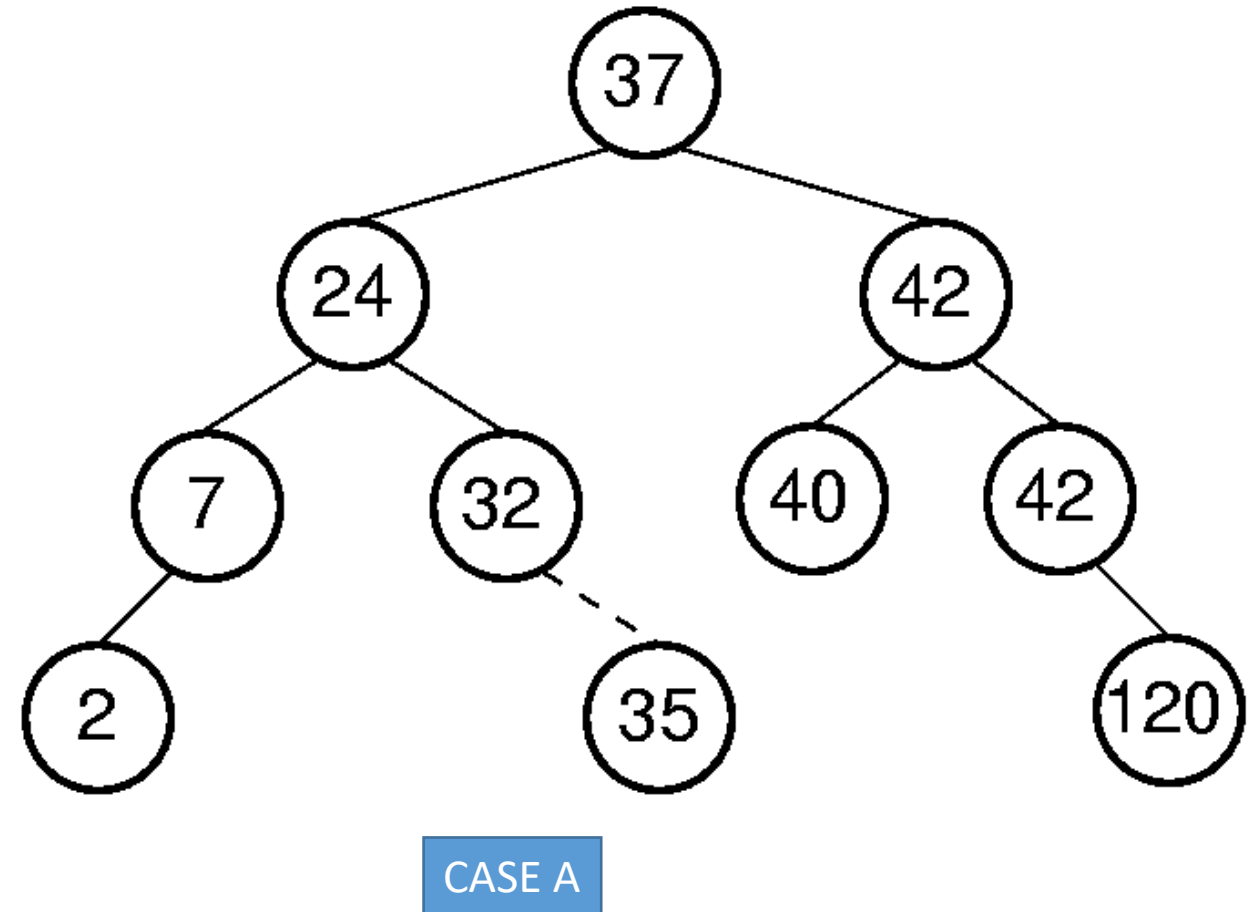
This empty node may hold values from ???

71 to 74



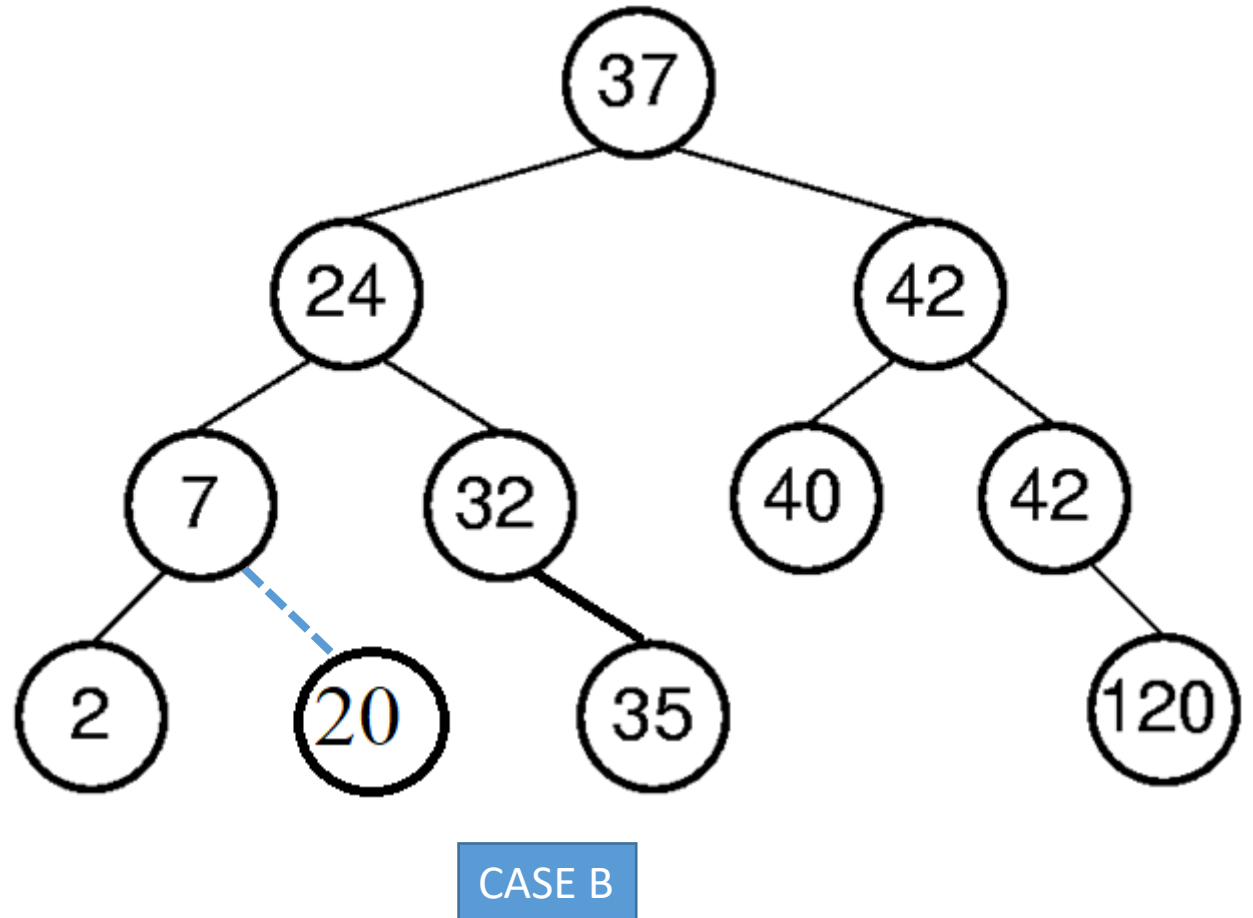
BST Insert

- We first find where that record (with key k) would have been if it were in the tree.
- This takes us to a node R'
 - Either R' is a leaf node [CASE A]
 - Or R' is an internal node with no child in the appropriate direction [CASE B]
- We then add a new node containing the new record as a child of R' .



BST Insert

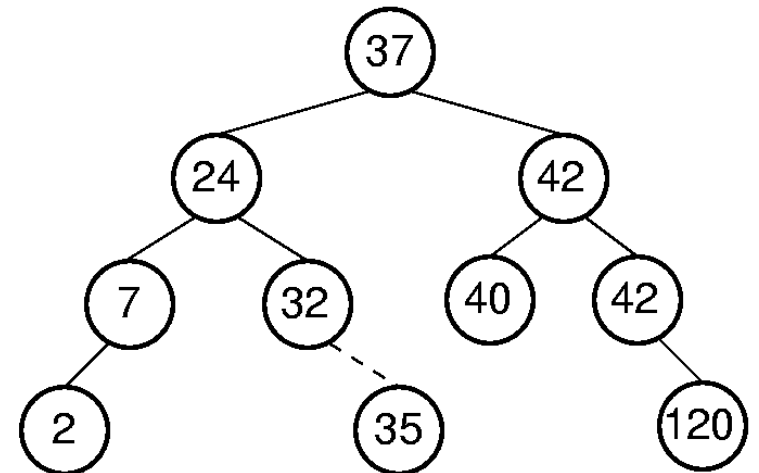
- We first find where that record (with key k) would have been if it were in the tree.
- This takes us to a node R'
 - Either R' is a leaf node [CASE A]
 - Or R' is an internal node with no child in the appropriate direction [CASE B]
- We then add a new node containing the new record as a child of R' .



BST Insert

```
template <typename Key, typename E>
BSTNode<Key, E>* BST<Key, E>::inserthelp(
    BSTNode<Key, E>* root, const Key& k, const E& it) {
    if (root == NULL)    // Empty tree: create node
        return new BSTNode<Key, E>(k, it, NULL, NULL);
    if (k < root->key())
        root->setLeft(inserthelp(root->left(), k, it));
    else root->setRight(inserthelp(root->right(), k, it));
    return root;        // Return tree with node inserted
}
```

```
// Insert a record into the tree.
// k Key value of the record.
// e The record to insert.
void insert(const Key& k, const E& e) {
    root = inserthelp(root, k, e);
    nodecount++;
}
```

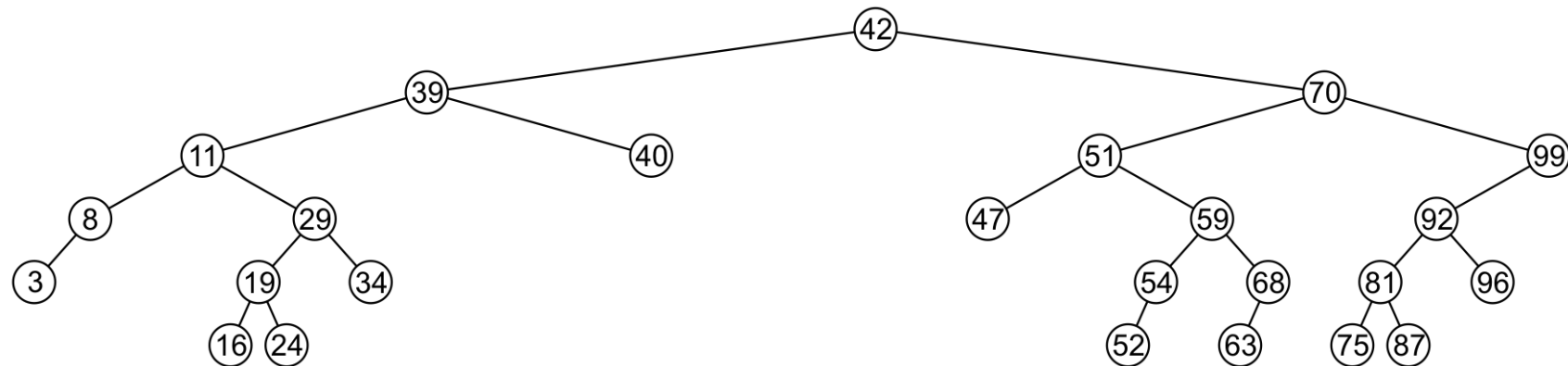


Erase/Remove/Delete

A node being erased is not always going to be a leaf node

There are three possible scenarios:

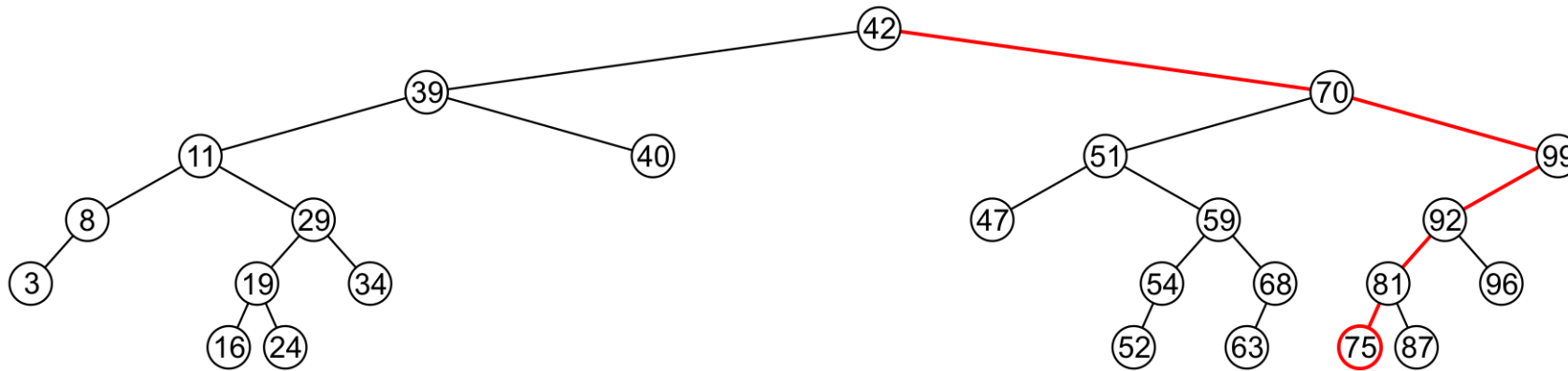
- The node is a leaf node,
- It has exactly one child, or
- It has two children (it is a full node)



Erase/Remove/Delete

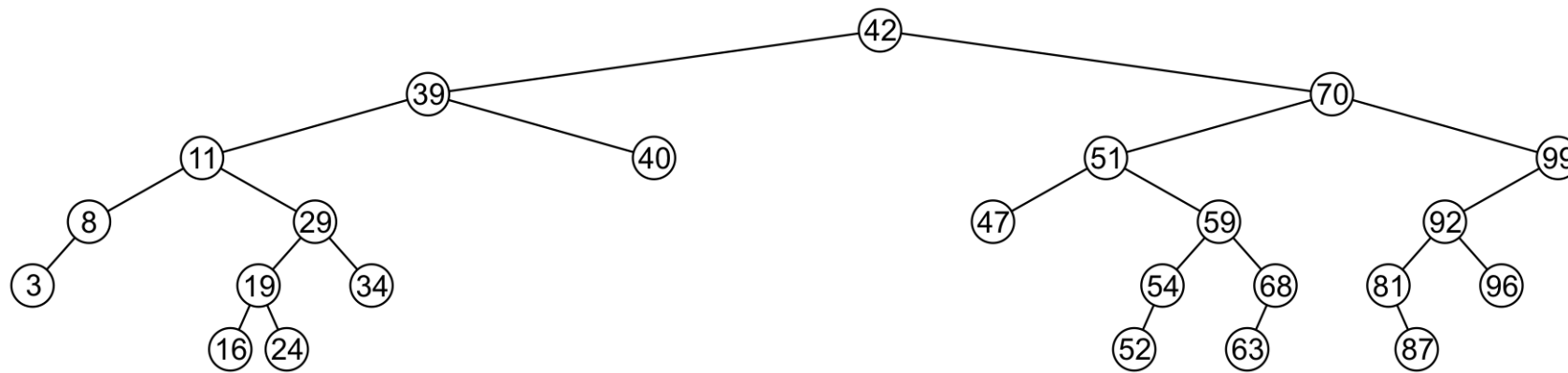
A leaf node simply must be removed and the appropriate member variable of the parent is set to nullptr

- Consider removing 75



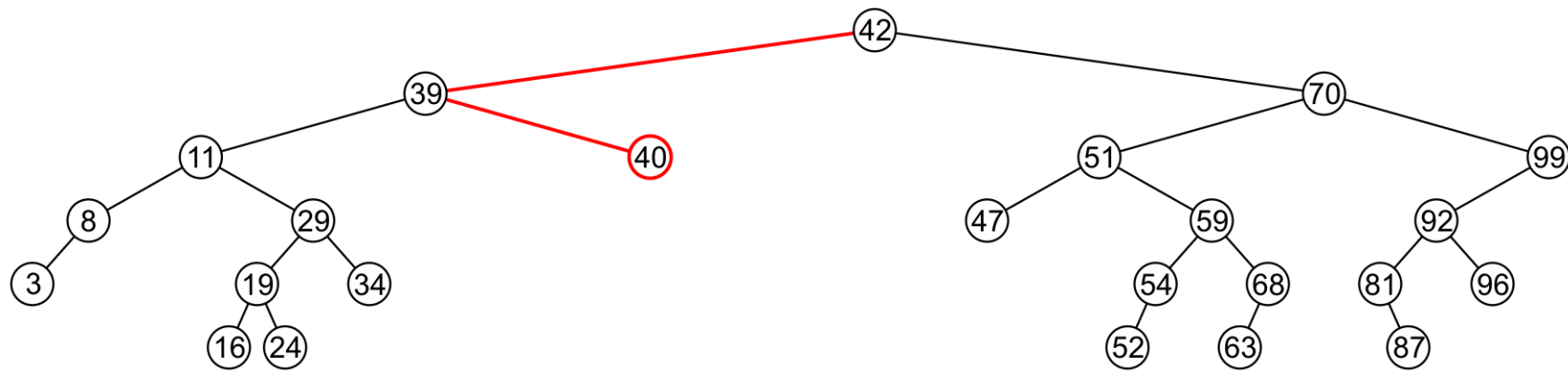
Erase/Remove/Delete

The node is deleted and `left_tree` of 81 is set to `nullptr`



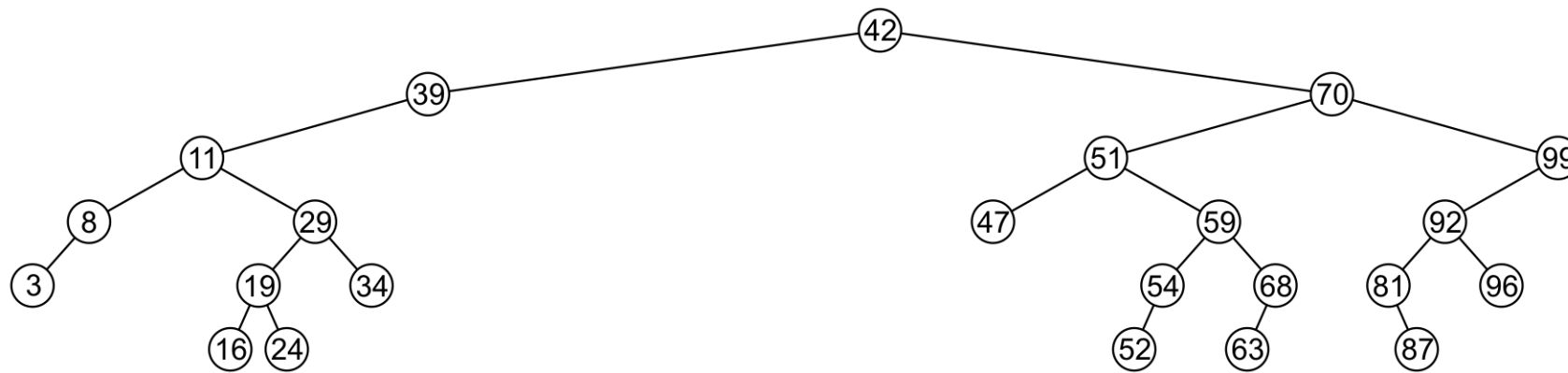
Erase/Remove/Delete

Erasing the node containing 40 is similar



Erase/Remove/Delete

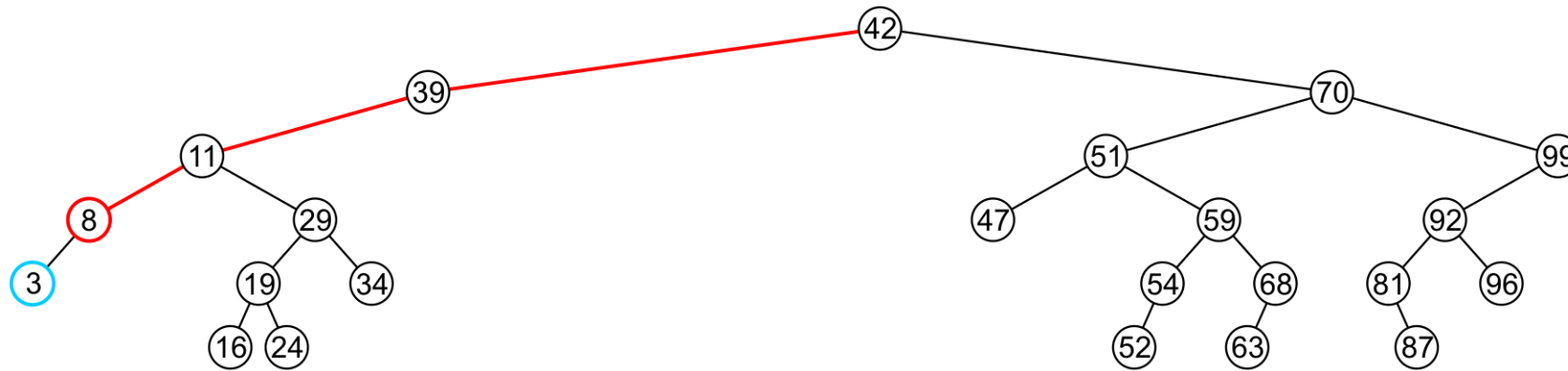
The node is deleted and `right_tree` of 39 is set to `nullptr`



Erase/Remove/Delete

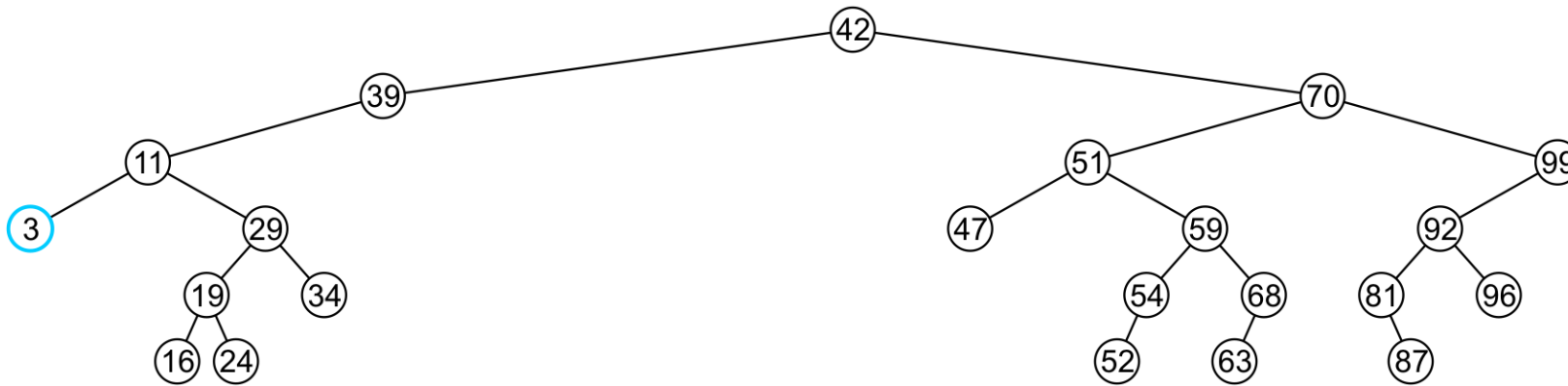
If a node has only one child, we can simply promote the subtree associated with the child

- Consider removing 8 which has one left child



Erase/Remove/Delete

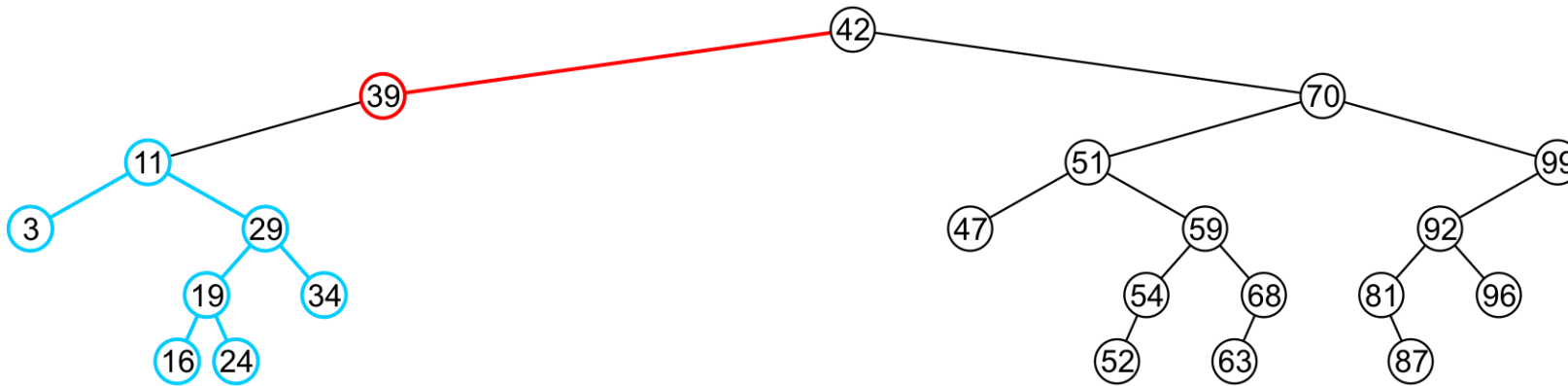
The node 8 is deleted and the `left_tree` of 11 is updated to point to 3



Erase/Remove/Delete

There is no difference in promoting a single node or a sub-tree

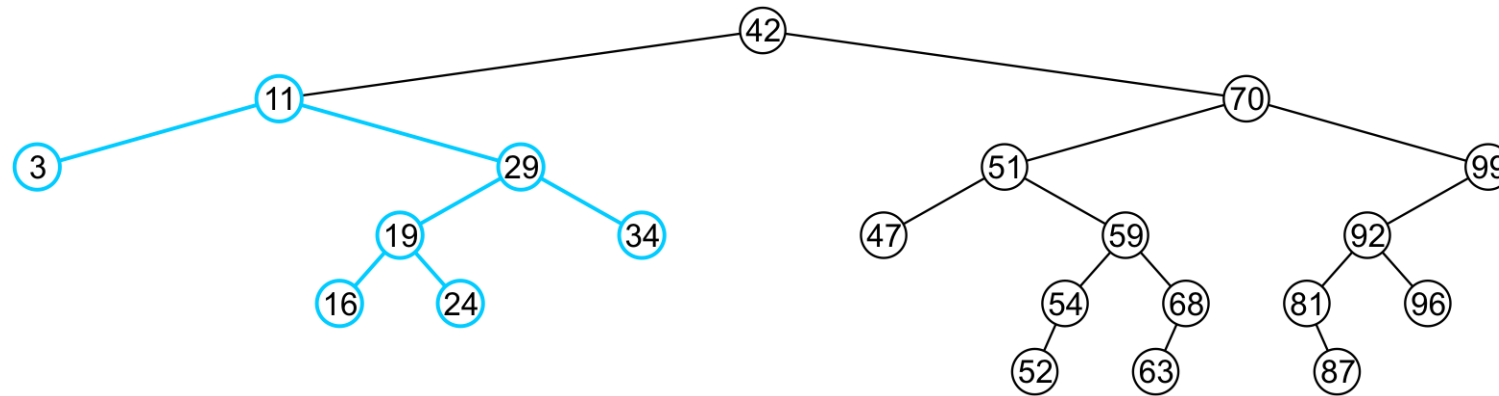
- To remove 39, it has a single child 11



Erase/Remove/Delete

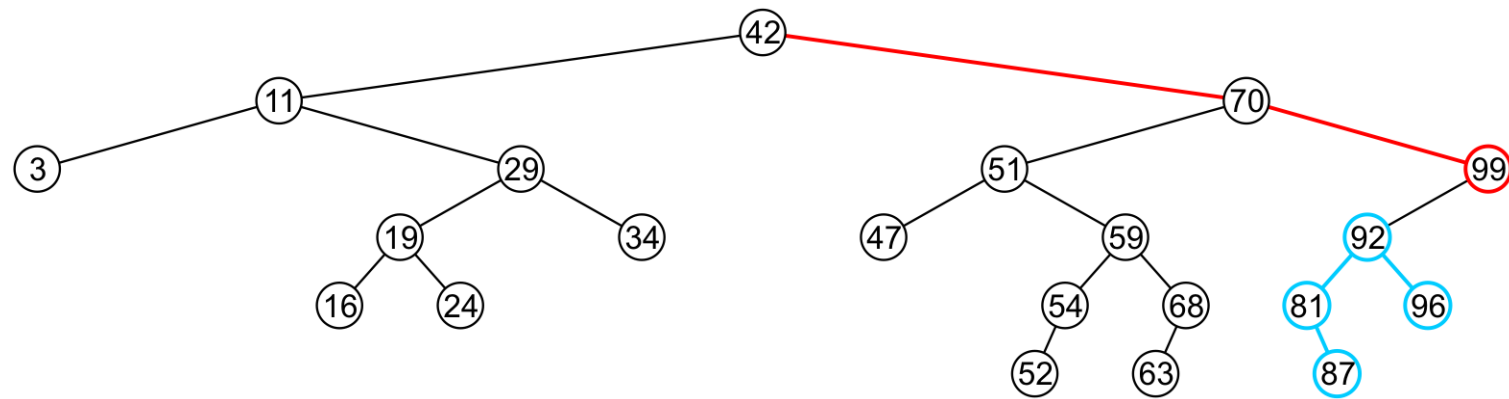
The node containing 39 is deleted and left_node of 42 is updated to point to 11

- Notice that order is still maintained



Erase/Remove/Delete

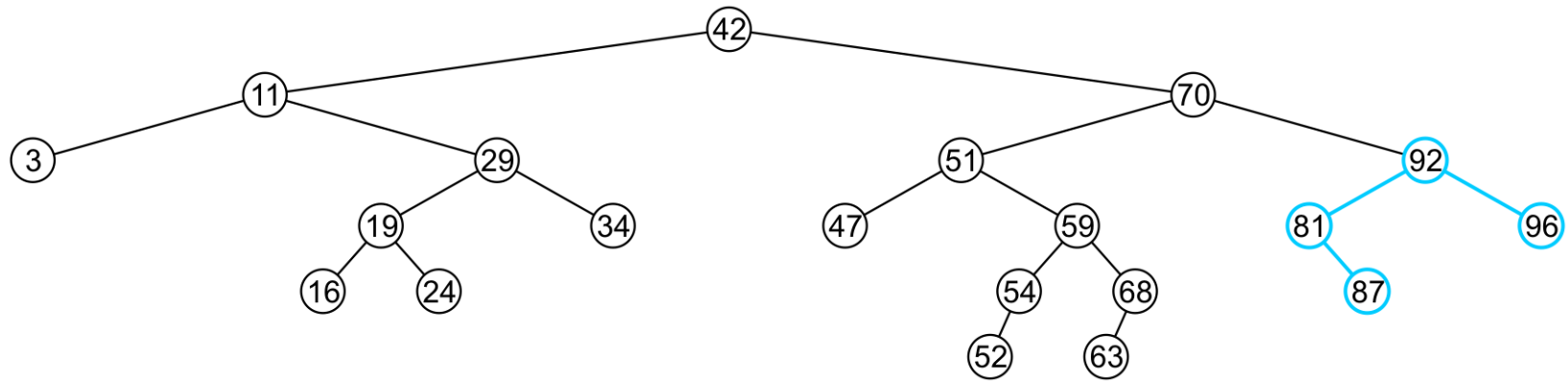
Consider erasing the node containing 99



Erase/Remove/Delete

The node is deleted and the left sub-tree is promoted:

- The member variable `right_tree` of 70 is set to point to 92
- Again, the order of the tree is maintained

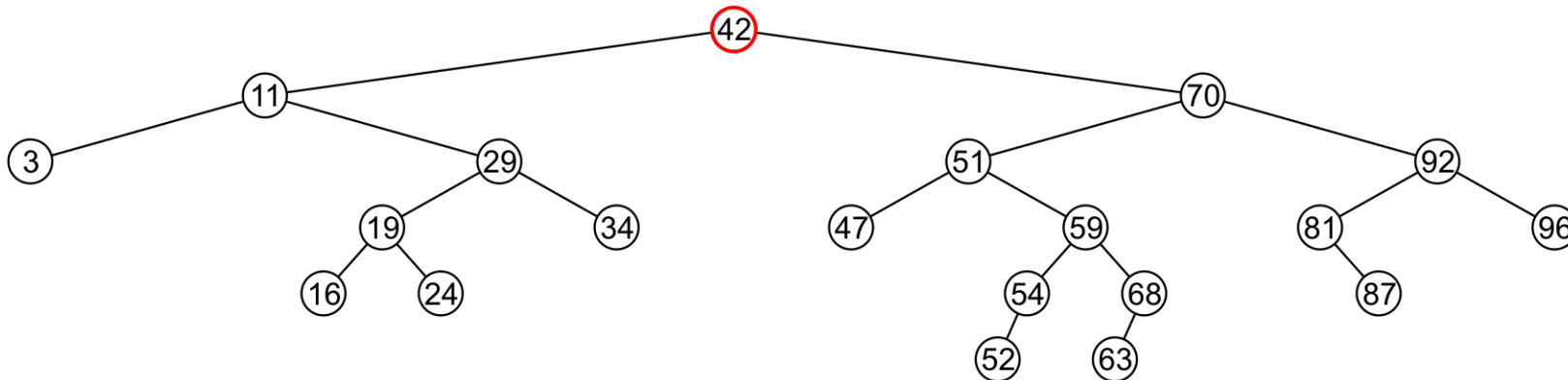


Erase/Remove/Delete

Finally, we will consider the problem of erasing a full node, *e.g.*, 42

We will perform two operations:

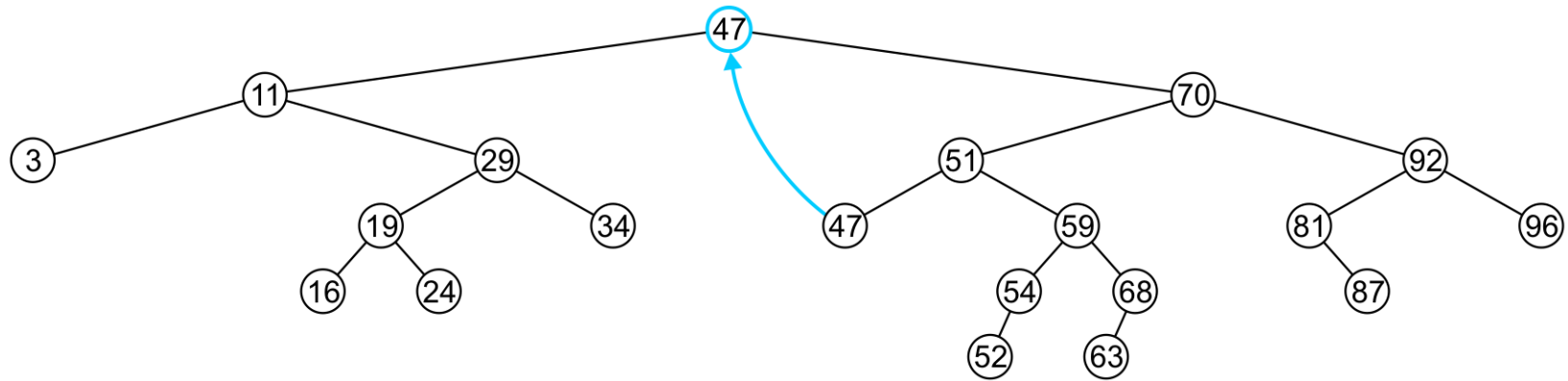
- Replace 42 with the minimum object in the right sub-tree
- Erase that object from the right sub-tree



Erase/Remove/Delete

In this case, we replace 42 with 47

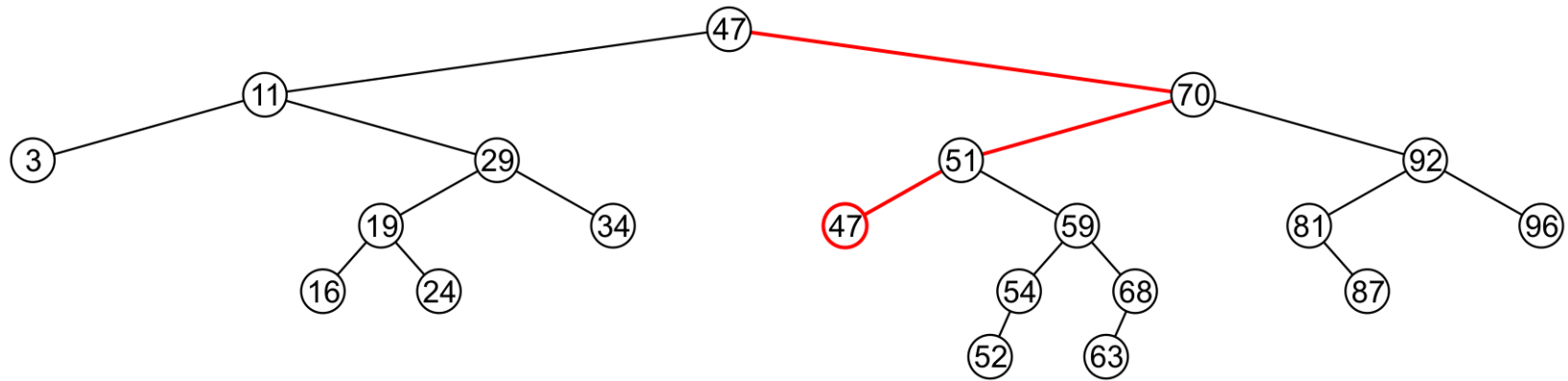
- We temporarily have two copies of 47 in the tree



Erase/Remove/Delete

We now recursively erase 47 from the right sub-tree

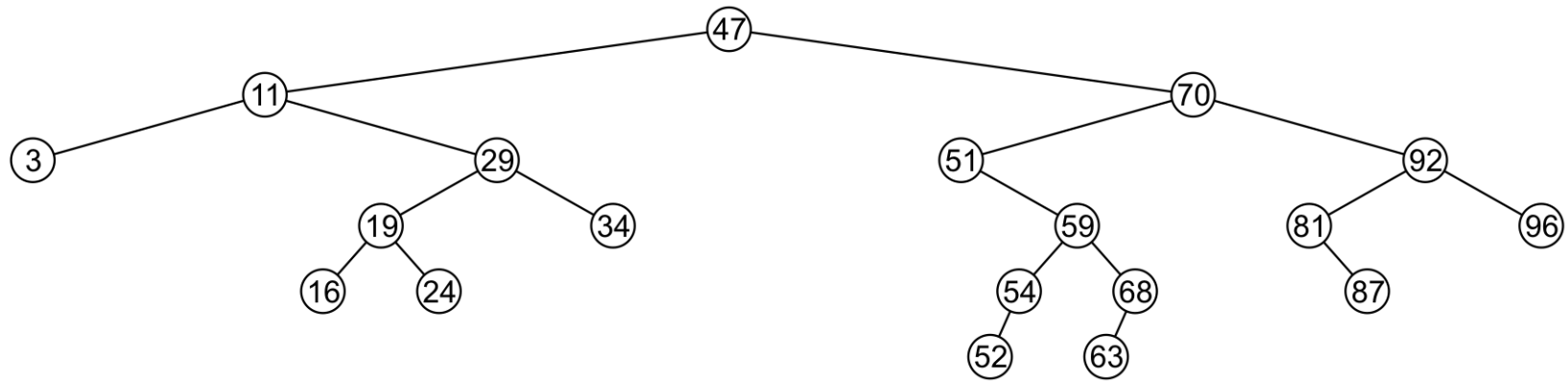
- We note that 47 is a leaf node in the right sub-tree



Erase/Remove/Delete

Leaf nodes are simply removed and `left_tree` of 51 is set to `nullptr`

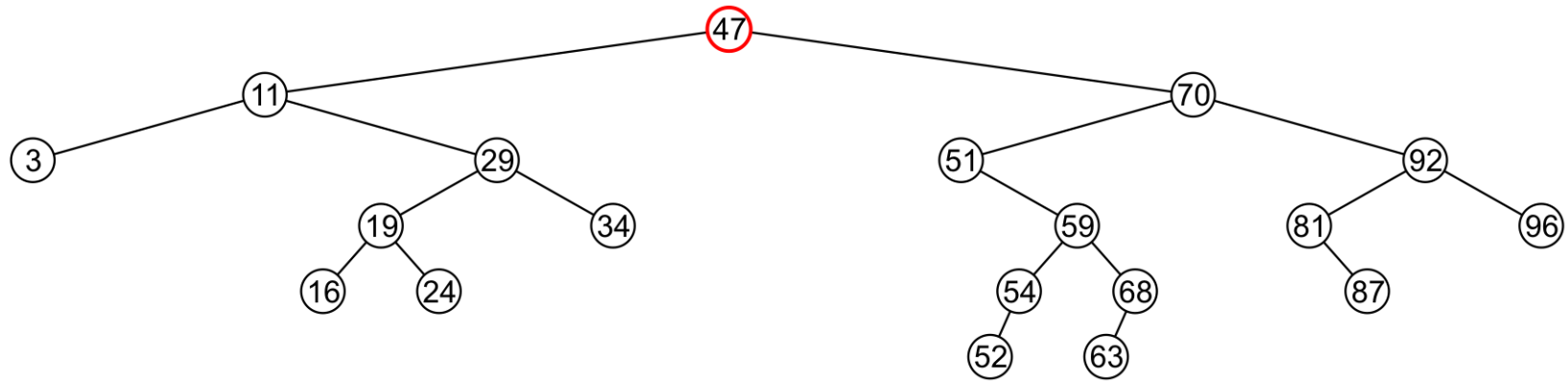
- Notice that the tree is still sorted:
47 was the least object in the right sub-tree



Erase/Remove/Delete

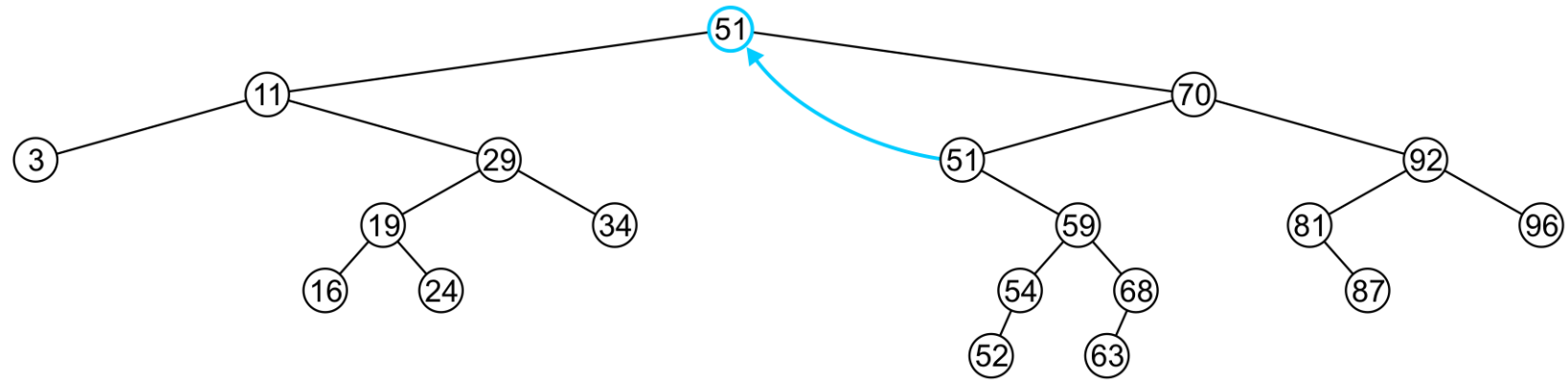
Suppose we want to erase the root 47 again:

- We must copy the minimum of the right sub-tree
- We could promote the maximum object in the left sub-tree and achieve similar results



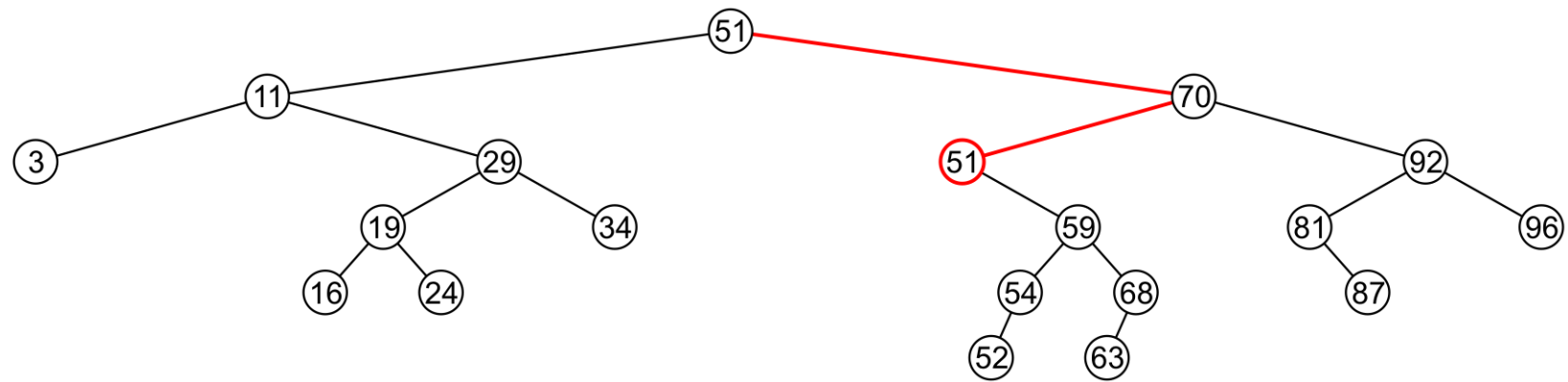
Erase/Remove/Delete

We copy 51 from the right sub-tree



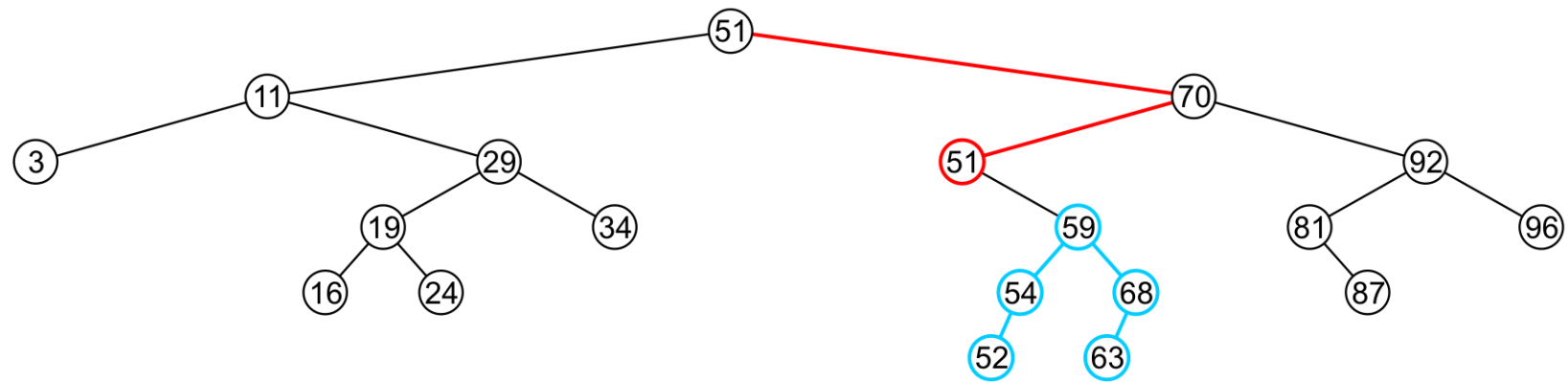
Erase/Remove/Delete

We must proceed by delete 51 from the right sub-tree



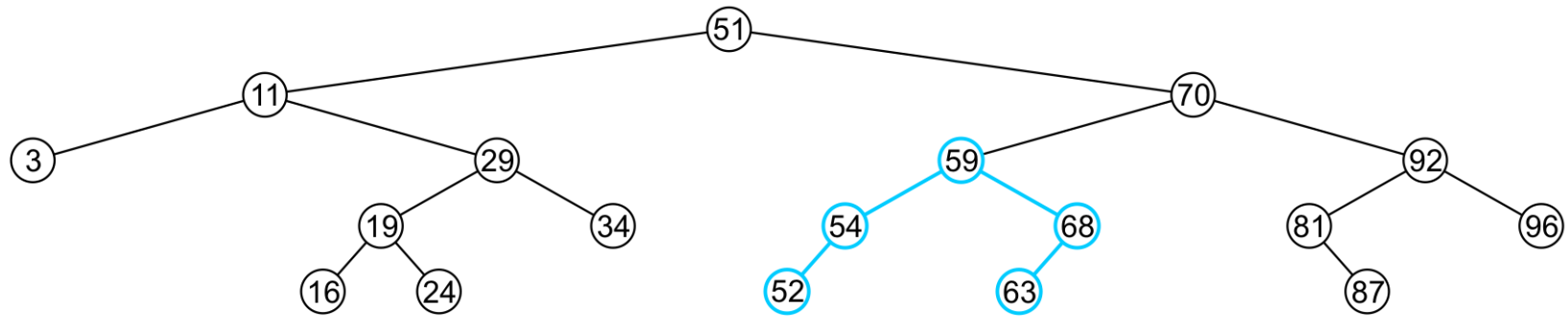
Erase/Remove/Delete

In this case, the node storing 51 has just a single child



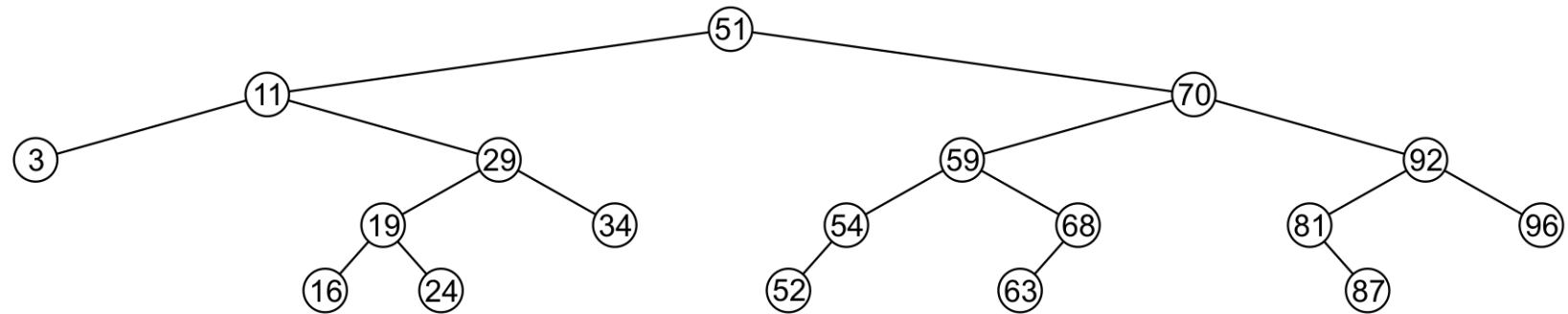
Erase/Remove/Delete

We delete the node containing 51 and assign the member variable `left_tree` of 70 to point to 59



Erase/Remove/Delete

Note that after seven removals, the remaining tree is still correctly sorted

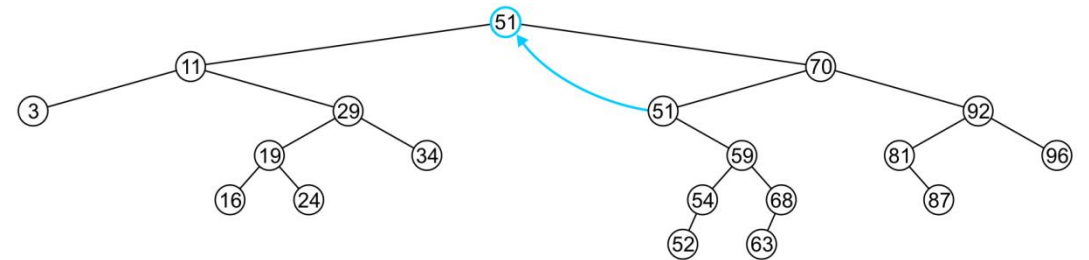
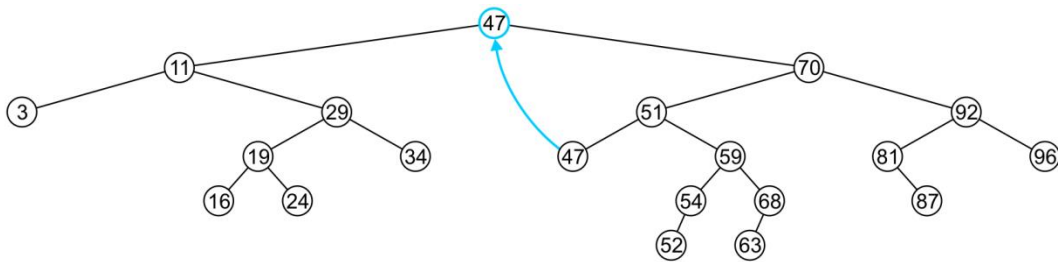


Erase/Remove/Delete

In the two examples of removing a full node, we promoted:

- A node with no children
- A node with right child

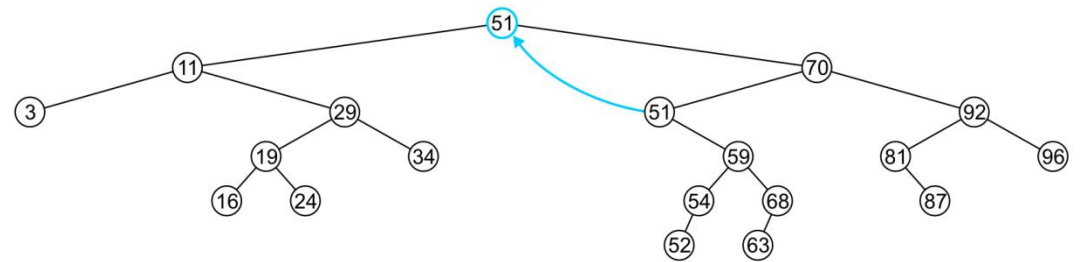
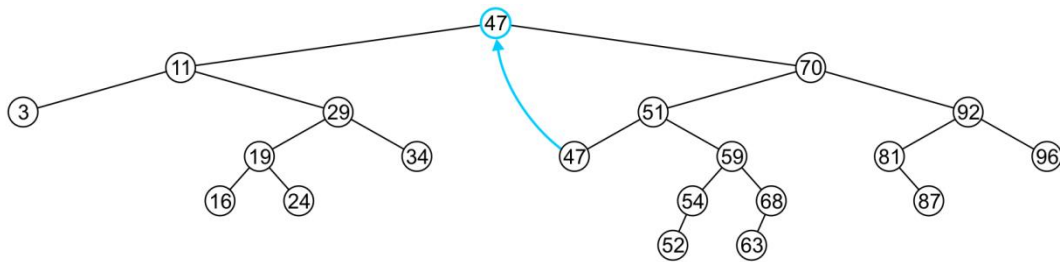
Is it possible, in removing a full node, to promote a child with two children?



Erase/Remove/Delete

Recall that we promoted the minimum value in the right sub-tree

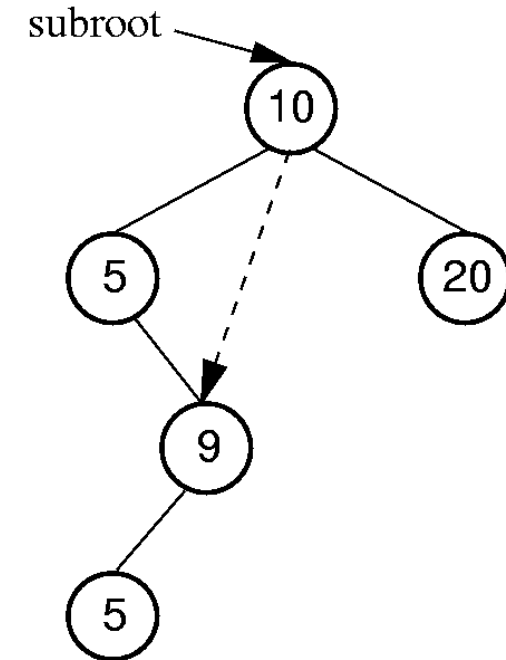
- If that node had a left sub-tree, that sub-tree would contain a smaller value



Get/Remove Minimum Value

```
private BSTNode<K,E>
    getmin(BSTNode<K,E> rt) {
    if (rt.left() == null)
        return rt;
    else return getmin(rt.left());
    }
```

```
private BSTNode<K,E>
    deletemin(BSTNode<K,E> rt) {
    if (rt.left() == null)
        return rt.right();
    else {
        rt.setLeft(deletemin(rt.left()));
        return rt;
    }
    }
```



BST Remove (1)

Sec. 5.4 Binary Search Trees

177

```
// Remove a node with key value k
// Return: The tree with the node removed
template <typename Key, typename E>
BSTNode<Key, E> BST<Key, E>::
removehelp(BSTNode<Key, E> rt, const Key& k) {
    if (rt == NULL) return NULL; // k is not in tree
    else if (k < rt->key())
        rt->setLeft(removehelp(rt->left(), k));
    else if (k > rt->key())
        rt->setRight(removehelp(rt->right(), k));
    else { // Found: remove it
        BSTNode<Key, E> temp = rt;
        if (rt->left() == NULL) { // Only a right child
            rt = rt->right(); // so point to right
            delete temp;
        }
        else if (rt->right() == NULL) { // Only a left child
            rt = rt->left(); // so point to left
            delete temp;
        }
        else { // Both children are non-empty
            BSTNode<Key, E> temp = getmin(rt->right());
            rt->setElement(temp->element());
            rt->setKey(temp->key());
            rt->setRight(deletehelp(rt->right()));
            delete temp;
        }
    }
    return rt;
}
```

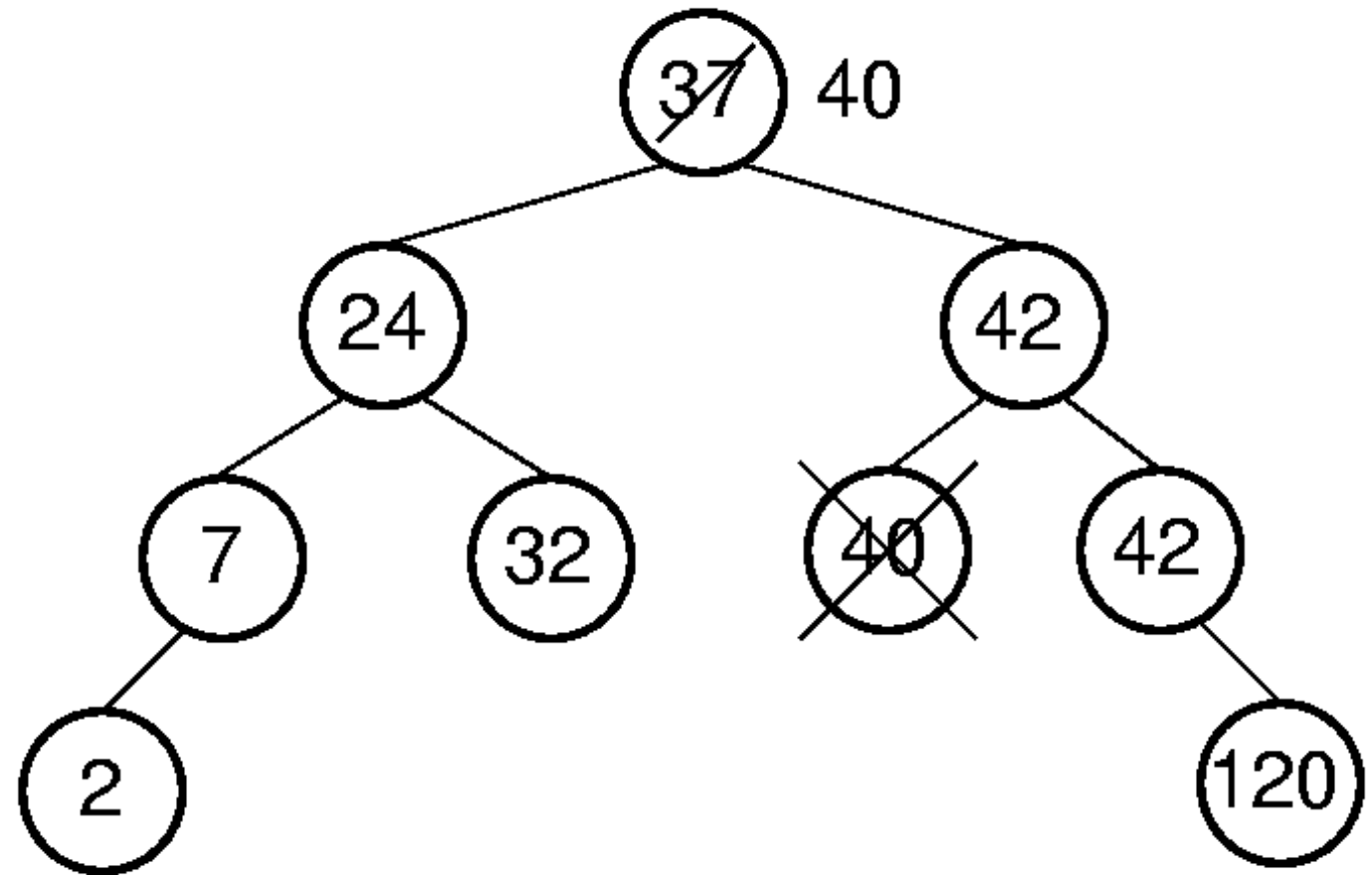
Figure 5.18 Implementation for the BST removehelp method.

likely to be good enough for this purpose), then each insertion will cost on average $\Theta(\log n)$, for a total cost of $\Theta(n \log n)$. However, if the records are inserted in order of increasing value, then the resulting tree will be a chain of height n . The cost of insertion in this case will be $\sum_{i=1}^n i = \Theta(n^2)$.

Traversing a BST costs $\Theta(n)$ regardless of the shape of the tree. Each node is visited exactly once, and each child pointer is followed exactly once.

Below are two example traversals. The first is member `clearhelp`, which returns the nodes of the BST to the freelist. Because the children of a node must be freed before the node itself, this is a postorder traversal.

```
template <typename Key, typename E>
void BST<Key, E>::
clearhelp(BSTNode<Key, E> root) {
    if (root == NULL) return;
    clearhelp(root->left());
    clearhelp(root->right());
    delete root;
}
```



Time Complexity of BST Operations

- Find: $\Theta(d)$
- Insert: $\Theta(d)$
- Delete: $\Theta(d)$
- d = depth of the deepest node in the BST, i.e., $\approx$ height of the tree.
- $d \approx \log n$ if tree is balanced.
- What is the worst case?

- This is why it is desirable to keep BSTs **balanced**
 - i.e., with least possible height.
- If a binary tree is balanced, then the height for a tree of n nodes is approximately $\log n$.
- However, if the tree is completely unbalanced, for example in the shape of a linked list, then the height for a tree with n nodes can be as great as n .

Thus, a balanced BST will in the average case have operations costing $\Theta(\log n)$, while a badly unbalanced BST can have operations in the worst case costing $\Theta(n)$.

Time Complexity of BST Operations

- Say, we construct a BST of n nodes by inserting records one at a time.
- If we are fortunate to have them arrive in an order that results in a balanced tree (a “random” order is likely to be good enough for this purpose), then each insertion will cost on average $\Theta(\log n)$
- Total cost will be $\Theta(n \log n)$.
- However, if the records are inserted in order of increasing value, then the resulting tree will be a chain of height n .
- The cost of construction in this case will be $\sum_{i=1}^n i = \Theta(n^2)$.
- Traversal will be $\Theta(n)$.